

Final Project Documentation for **pokemondb**
Timothy Jeon

Created DB **pokemondb**

```
[postgres=# CREATE DATABASE pokemondb;
CREATE DATABASE
[postgres=# \c pokemondb
You are now connected to database "pokemondb" as user "postgres".
```

Created 6 main tables

```
CREATE TABLE sets (
    set_id TEXT PRIMARY KEY,
    set_name TEXT NOT NULL,
    release_date DATE
);

CREATE TABLE artists (
    artist_id SERIAL PRIMARY KEY,
    artist TEXT UNIQUE NOT NULL
);

CREATE TABLE rarities (
    rarity_id SERIAL PRIMARY KEY,
    rarity TEXT UNIQUE NOT NULL
);

CREATE TABLE cards (
    card_id TEXT PRIMARY KEY,
    name TEXT NOT NULL,
    supertype TEXT,
    subtypes TEXT,
    set_id TEXT REFERENCES sets(set_id),
    artist_id INT REFERENCES artists(artist_id),
    rarity_id INT REFERENCES rarities(rarity_id)
);

CREATE TABLE variants (
    variant_id SERIAL PRIMARY KEY,
    card_id TEXT REFERENCES cards(card_id),
    variant TEXT NOT NULL
);

CREATE TABLE card_prices (
    price_id SERIAL PRIMARY KEY,
    variant_id INT REFERENCES variants(variant_id),
    market_price NUMERIC(10,2) NOT NULL,
```

```
date_observed DATE NOT NULL
);
```

One-to-Many relationships

sets -> cards

- one set is able to contain many cards
artists -> cards
- one artist is able to create many cards
rarities -> cards
- one rarity can be applied to many cards
cards -> variants
- one card can be many different variants
variants -> card_prices
- one variant can be many different prices

Many-to-One relationships

cards -> sets

- many cards are in one set
cards -> artists
- many cards can share a single artist
cards -> rarities
- many cards can have the same rarity
variants -> cards
- many variants belong to a single card
card_prices -> variants
- many prices can belong to one variant

```
pokemondb=# \d
List of relations
Schema | Name | Type | Owner
-----+-----+-----+-----
public | artists | table | postgres
public | artists_artist_id_seq | sequence | postgres
public | card_prices | table | postgres
public | card_prices_price_id_seq | sequence | postgres
public | cards | table | postgres
public | rarities | table | postgres
public | rarities_rarity_id_seq | sequence | postgres
public | sets | table | postgres
public | variants | table | postgres
public | variants_variant_id_seq | sequence | postgres
(10 rows)
```

Used OLD dataset called `pokemon_data` that looks like this.

card_id	name	variant	rarity	supertype	subtypes	set_id	set_name	release_date	artist	market_price	date_observed
hgss4-1	Aggron	holofoil	Rare Holo	Pokémon	Stage 2	hgss4	HS—Triumphant	2010-11-03	Kagemaru Himeno	5.99	2026-01-25
hgss4-1	Aggron	reverseHolofoil	Rare Holo	Pokémon	Stage 2	hgss4	HS—Triumphant	2010-11-03	Kagemaru Himeno	4.73	2026-01-25
xy5-1	Weedle	normal	Common	Pokémon	Basic	xy5	Primal Clash	2015-02-04	Midori Harada	0.12	2026-01-25
xy5-1	Weedle	reverseHolofoil	Common	Pokémon	Basic	xy5	Primal Clash	2015-02-04	Midori Harada	0.49	2026-01-25
pl1-1	Ampharos	holofoil	Rare Holo	Pokémon	Stage 2	pl1	Platinum	2009-02-11	Atsuko Nishida	31.16	2026-01-25
pl1-1	Ampharos	reverseHolofoil	Rare Holo	Pokémon	Stage 2	pl1	Platinum	2009-02-11	Atsuko Nishida	15.29	2026-01-25
dp3-1	Ampharos	holofoil	Rare Holo	Pokémon	Stage 2	dp3	Secret Wonders	2007-11-01	Kouki Saitou	25.1	2026-01-25
dp3-1	Ampharos	reverseHolofoil	Rare Holo	Pokémon	Stage 2	dp3	Secret Wonders	2007-11-01	Kouki Saitou	24.43	2026-01-25
det1-1	Bulbasaur	holofoil	Common	Pokémon	Basic	det1	Detective Pikachu	2019-04-05	MPC Film	0.61	2026-01-25
dv1-1	Dratini	holofoil	Rare Holo	Pokémon	Basic	dv1	Dragon Vault	2012-10-05	Masakazu Fukuda	2.4	2026-01-25
mcd19-1	Caterpie	holofoil	Unknown	Pokémon	Basic	mcd19	McDonald's Collection 2019	2019-10-15	Sekio	10.57	2026-01-25
pl3-1	Absol G	holofoil	Rare Holo	Pokémon	Basic,SP	pl3	Supreme Victors	2009-08-19	Yusuke Ishikawa	17.23	2026-01-25
pl3-1	Absol G	reverseHolofoil	Rare Holo	Pokémon	Basic,SP	pl3	Supreme Victors	2009-08-19	Yusuke Ishikawa	13.41	2026-01-25
ex12-1	Aerodactyl	holofoil	Rare Holo	Pokémon	Stage 1	ex12	Legend Maker	2006-02-01	Hajime Kusajima	60.95	2026-01-25
ex12-1	Aerodactyl	reverseHolofoil	Rare Holo	Pokémon	Stage 1	ex12	Legend Maker	2006-02-01	Hajime Kusajima	59.83	2026-01-25
ex3-1	Absol	holofoil	Rare Holo	Pokémon	Basic	ex3	Dragon	2003-11-24	Naoyo Kimura	395	2026-01-25
ex3-1	Absol	reverseHolofoil	Rare Holo	Pokémon	Basic	ex3	Dragon	2003-11-24	Naoyo Kimura	89.76	2026-01-25

Temporarily created table called `poke_data` to effectively import all dataset entries from `pokemon_data - Sheet1.csv` into this table so I could easily insert the right data into the other tables.

```
CREATE TABLE poke_data (  
  card_id TEXT,  
  name TEXT,  
  variant TEXT,  
  rarity TEXT,  
  supertype TEXT,  
  subtypes TEXT,  
  set_id TEXT,  
  set_name TEXT,  
  release_date DATE,  
  artist TEXT,  
  market_price NUMERIC(10,2),  
  date_observed DATE  
);
```

```
INSERT INTO sets(set_id, set_name, release_date)  
SELECT DISTINCT set_id, set_name, release_date  
FROM poke_data  
ON CONFLICT (set_id) DO NOTHING;
```

```
INSERT INTO artists (artist)  
SELECT DISTINCT artist FROM poke_data  
WHERE artist IS NOT NULL;
```

```
INSERT INTO rarities (rarity)  
SELECT DISTINCT rarity FROM poke_data  
WHERE rarity is NOT NULL;
```

```
INSERT INTO cards ( card_id, name, supertype, subtypes, set_id, artist_id,  
rarity_id)
```

```

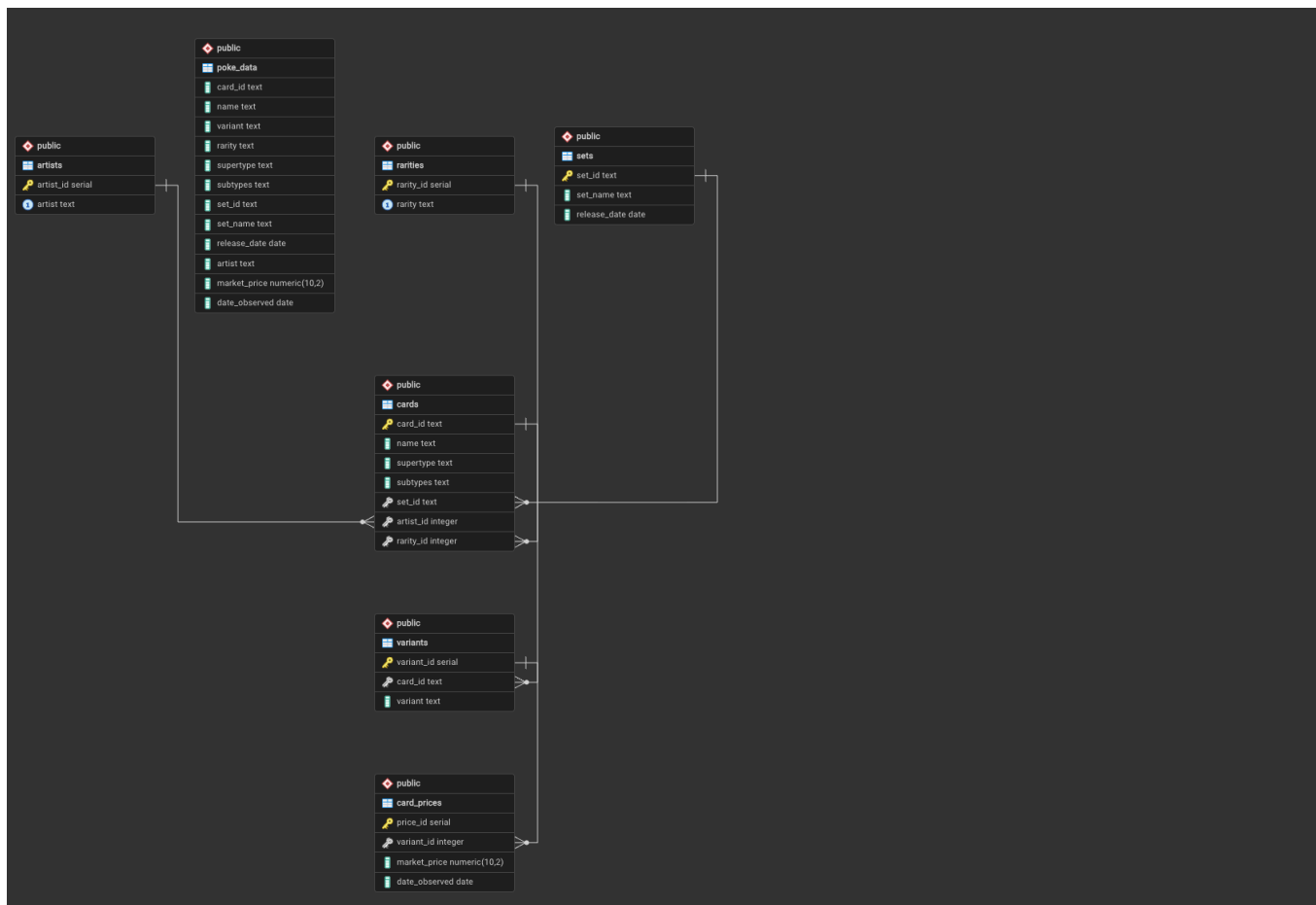
SELECT DISTINCT
    pd.card_id,
    pd.name,
    pd.supertype,
    pd.subtypes,
    pd.set_id,
    a.artist_id,
    r.rarity_id
FROM poke_data pd
LEFT JOIN artists a ON pd.artist = a.artist
LEFT JOIN rarities r ON pd.rarity = r.rarity;

INSERT INTO variants (card_id, variant)
SELECT DISTINCT card_id, variant FROM poke_data;

INSERT INTO card_prices (variant_id, market_price, date_observed)
SELECT
    v.variant_id,
    pd.market_price,
    pd.date_observed
FROM poke_data pd
JOIN variants v ON pd.card_id = v.card_id AND pd.variant = v.variant;

```

ERD diagram



Every single query I made

--- Count all the pokemon cards in each set

```
SELECT set_name, COUNT(*) AS card_count
FROM cards c
JOIN sets s ON c.set_id = s.set_id
GROUP BY set_name
ORDER BY card_count DESC;
```

--- Returns all artists

```
SELECT artist FROM artists;
```

--- Returns 10 artists

```
SELECT artist FROM artists LIMIT 10;
```

--- Finds total card value by each artist

--- IMPLEMENTED VIEW to showcase

```
CREATE VIEW artist_total_value AS
SELECT artist, SUM(market_price) AS total_value
FROM cards c
JOIN artists a ON c.artist_id = a.artist_id
JOIN variants v ON c.card_id = v.card_id
JOIN card_prices cp ON v.variant_id = cp.variant_id
GROUP BY a.artist
ORDER BY total_value DESC;
```

```
SELECT * FROM artist_total_value;
```

--- Get's total card value for each artist but return only those who have total values between \$1000 and \$5000

```
WITH artist_value AS(
SELECT artist, SUM(market_price) AS total_value FROM cards c
JOIN artists a ON c.artist_id = a.artist_id
JOIN sets s ON c.set_id = s.set_id
JOIN variants v ON c.card_id = v.card_id
JOIN card_prices cp ON v.variant_id = cp.variant_id
GROUP BY artist
)
SELECT artist, total_value FROM artist_value WHERE total_value >= 1000 AND
total_value <= 5000 ORDER BY total_value;
```

--- Get total value of cards for each set

```
SELECT set_name, SUM(market_price) AS total_value FROM cards c
JOIN sets s ON c.set_id = s.set_id
JOIN variants v ON c.card_id = v.card_id
JOIN card_prices cp ON v.variant_id = cp.variant_id
GROUP BY set_name ORDER BY total_value DESC;
```

--- Same as code above but adds LIMIT 5 to get the 5 sets with the most total value

```
SELECT set_name, SUM(market_price) AS total_value FROM cards c
JOIN sets s ON c.set_id = s.set_id
JOIN variants v ON c.card_id = v.card_id
```

```
JOIN card_prices cp ON v.variant_id = cp.variant_id
GROUP BY set_name ORDER BY total_value DESC LIMIT 5;
```

--- Gets all names of different Pokemon done by a single artist

```
SELECT DISTINCT name FROM cards c
JOIN artists a ON c.artist_id = a.artist_id
WHERE artist = 'Yuka Morii';
```

--- Gets information for all Rayquaza cards

```
SELECT * FROM cards WHERE name LIKE 'Rayquaza%';
```

--- Returns all Rayquaza cards that have the VMAX variant

```
SELECT * FROM cards WHERE name LIKE 'Rayquaza_VMAX';
```

--- Function that returns all cards of a single Pokemon including the set, variant, and its price.

--- In this example, I wanted to get all Rayquaza cards in existence.

```
CREATE OR REPLACE FUNCTION search_pokemon(pokemon_name TEXT)
```

```
RETURNS TABLE (
    name TEXT,
    set_name TEXT,
    variant TEXT,
    market_price NUMERIC
)
```

```
AS $$
```

```
BEGIN
```

```
    RETURN QUERY
```

```
    SELECT
```

```
        c.name,
        s.set_name,
        v.variant,
        cp.market_price
```

```
    FROM cards c
```

```
    JOIN sets s ON c.set_id = s.set_id
```

```
    JOIN variants v ON c.card_id = v.card_id
```

```
    JOIN card_prices cp ON v.variant_id = cp.variant_id
```

```
    WHERE c.name LIKE pokemon_name || '%';
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

```
SELECT * FROM search_pokemon('Rayquaza');
```

--- Gets the most expensive card for each set. Used a CTE to also include the name of the Pokemon.

```
WITH ranked AS (
```

```
    SELECT
```

```
        s.set_name,
        c.name,
        v.variant,
        cp.market_price,
        ROW_NUMBER() OVER (
```

```
            PARTITION BY s.set_name
```

```
            ORDER BY cp.market_price DESC
```

```

        ) AS rank
    FROM cards c
    JOIN sets s ON c.set_id = s.set_id
    JOIN variants v ON c.card_id = v.card_id
    JOIN card_prices cp ON v.variant_id = cp.variant_id
)
SELECT set_name, name, variant, market_price
FROM ranked
WHERE rank = 1
ORDER BY market_price DESC;

```

--- Check to see if there is an error with data
 --- Looks like the set Paldea Evolved is incomplete as values for a lot of cards don't make sense

```

SELECT set_name, name, market_price
FROM cards c
JOIN sets s ON c.set_id = s.set_id
JOIN variants v ON c.card_id = v.card_id
JOIN card_prices cp ON v.variant_id = cp.variant_id
WHERE set_name = 'Paldea Evolved';

```

--- Black Bolt data looks much more accurate

```

SELECT set_name, name, market_price
FROM cards c
JOIN sets s ON c.set_id = s.set_id
JOIN variants v ON c.card_id = v.card_id
JOIN card_prices cp ON v.variant_id = cp.variant_id
WHERE set_name = 'Black Bolt';

```

--- Shows price difference between most and least expensive variants of a card

```

SELECT
name,
set_name,
MAX(market_price), MIN(market_price), MAX(market_price) - MIN(market_price)
AS price_range
FROM cards c
JOIN sets s ON c.set_id = s.set_id
JOIN variants v ON c.card_id = v.card_id
JOIN card_prices cp ON v.variant_id = cp.variant_id
GROUP BY c.name, set_name
ORDER BY price_range DESC;

```

--- Gets card price average > 50

--- uses HAVING

```

SELECT
c.name,
AVG(market_price) AS avg_price
FROM cards c
JOIN variants v ON c.card_id = v.card_id
JOIN card_prices cp ON v.variant_id = cp.variant_id

```

```

GROUP BY c.name
HAVING AVG(cp.market_price) > 50;

--- INNER JOIN where it only gets cards with their set names
SELECT name, set_name FROM cards c INNER JOIN sets s ON c.set_id = s.set_id;

-- LEFT JOIN
-- SHOWS ALL CARDS EVEN IF THEY DON'T HAVE VARIANTS
SELECT name, variant FROM cards c LEFT JOIN variants v ON c.card_id =
v.card_id;

-- RIGHT JOIN
-- SHOWS ALL VARIANTS EVEN IF CARD IS MISSING
SELECT name, variant FROM cards c RIGHT JOIN variants v ON c.card_id =
v.card_id;

-- CROSS JOIN
-- PAIR EVERY CARD WITH THEIR RARITY
SELECT name, rarity FROM cards c CROSS JOIN rarities r;

--- CONCAT_WS to combine name, variant, and the price into a string
separated by /
SELECT CONCAT_WS(' / ', name, variant, market_price) AS card_info
FROM cards c
JOIN variants v ON c.card_id = v.card_id
JOIN card_prices cp ON v.variant_id = cp.variant_id;

--- Returns info of all Pokemon cards that has 4 letters
SELECT * FROM cards WHERE LENGTH(name) = 4;

--- Returns name of Pokemon but reversed
SELECT name, REVERSE(name) AS name_reversed FROM cards;

--- Replaces reverseHolofoil to Reverse Holofoil for Pokemon with this
variant
SELECT name, variant, REPLACE(variant, 'reverseHolofoil', 'Reverse
Holofoil') AS new_variant
FROM cards c
JOIN variants v ON c.card_id = v.card_id;

--- Procedure that actually updates reverseHolofoil to Reverse Holofoil to
make the wording more clean
CREATE OR REPLACE PROCEDURE fix_reverse_holofoil()
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE variants
    SET variant = 'Reverse Holofoil'
    WHERE variant = 'reverseHolofoil';
END;
$$;

```



```

CALL fix_reverse_holofoil();
---NOTE: all the data variants that changed go to the back of table so run
this below to see everything in order
SELECT * FROM variants ORDER BY variant_id;

--- Get's first two and last two letters of Pokemon card name
SELECT name, LEFT(name, 2) AS first_two, RIGHT(name, 2) AS last_two FROM
cards;

--- Same thing as code above but adds - between the shortened names
SELECT name, CONCAT_WS('-', LEFT(name, 2), RIGHT(name, 2)) AS shortened_name
FROM cards;

--- Another way to get the first two and last two letters of a Pokemon card
name using SUBSTR instead of LEFT and RIGHT
SELECT name, CONCAT_WS('-', SUBSTR(name, 1, 2), SUBSTR(name, LENGTH(name) -
1)) AS shortened_name FROM cards;

--- Returns all pokemon with EX by using POSITION
SELECT name FROM cards WHERE POSITION('EX' IN name) > 0;

--- Function where user can type set and get the most expensive cards by
entering a count of how many they want to see within a set
CREATE OR REPLACE FUNCTION get_top_cards_by_set(
    set_input VARCHAR(50),
    n INT
)
RETURNS TABLE (
    name TEXT,
    variant TEXT,
    market_price NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT
        c.name,
        v.variant,
        cp.market_price
    FROM cards c
    JOIN sets s ON c.set_id = s.set_id
    JOIN variants v ON c.card_id = v.card_id
    JOIN card_prices cp ON v.variant_id = cp.variant_id
    WHERE s.set_name = set_input
    ORDER BY cp.market_price DESC
    LIMIT n;
END;
$$;

```

```

SELECT * FROM get_top_cards_by_set('Destined Rivals', 7);

--- Gets most expensive card out of every other card
--- Can't use GROUP BY if I want one row only
SELECT name, set_name, variant, market_price AS most_expensive_price
FROM cards c
JOIN sets s ON c.set_id = s.set_id
JOIN variants v ON c.card_id = v.card_id
JOIN card_prices cp ON v.variant_id = cp.variant_id
ORDER BY cp.market_price DESC LIMIT 1;

--- Get's total sum of price of cards for a particular set (in this case
Lost Origin)
--- Could have made this a function
SELECT set_name, SUM(market_price) as total_value
FROM cards c
JOIN sets s ON c.set_id = s.set_id
JOIN variants v ON c.card_id = v.card_id
JOIN card_prices cp ON v.variant_id = cp.variant_id
WHERE set_name = 'Lost Origin' GROUP BY set_name;

```

Queries with New Features

Query below uses both `CTE` and window function `ROW_NUMBER()`.

```

--- Gets the most expensive card for each set. Used a CTE to also include
the name of the Pokemon.
WITH ranked AS (
    SELECT
        s.set_name,
        c.name,
        v.variant,
        cp.market_price,
        ROW_NUMBER() OVER (
            PARTITION BY s.set_name
            ORDER BY cp.market_price DESC
        ) AS rank
    FROM cards c
    JOIN sets s ON c.set_id = s.set_id
    JOIN variants v ON c.card_id = v.card_id
    JOIN card_prices cp ON v.variant_id = cp.variant_id
)
SELECT set_name, name, variant, market_price
FROM ranked
WHERE rank = 1
ORDER BY market_price DESC;

```

Query below uses `MAX()` and `MIN()` aggregate functions and subtracts from each other to find difference.

```

--- Shows price difference between most and least expensive variants of a
card
SELECT
name,
set_name,
MAX(market_price), MIN(market_price), MAX(market_price) - MIN(market_price)
AS price_range
FROM cards c
JOIN sets s ON c.set_id = s.set_id
JOIN variants v ON c.card_id = v.card_id
JOIN card_prices cp ON v.variant_id = cp.variant_id
GROUP BY c.name, set_name
ORDER BY price_range DESC;

```

Uses `CONCAT_WS()` and `SUBSTR()` to get the first and last two letters of a Pokemon and placing `-` between them.

```

--- Another way to get the first two and last two letters of a Pokemon card
name using SUBSTR instead of LEFT and RIGHT
SELECT name, CONCAT_WS('-', SUBSTR(name, 1, 2), SUBSTR(name, LENGTH(name) -
1)) AS shortened_name FROM cards;

```